

Spoke — README and AI usage

Author: Shubhangini

Repo: <https://github.com/shubhangini67/spoke-llm-practice>

Live: <https://spoke-llm-practice.vercel.app>

2-day CipherSchools LLD practice MVP. Combined file for the assignment form. Full diagrams and screenshots are in the GitHub README.

Product

Spoke is a small web app for practicing LLD interviews. You pick a problem, lock clarifying questions, name types, walk one use case, defend a rejected path, submit, and get feedback that does not depend on matching a blog's class names.

Quick demo: **Parking Lot → Start attempt → Load example → Submit**. Then try **Revise for the follow-up**.

Stack	Next.js (App Router), TypeScript, JSON file for attempts
Problems	Vending Machine, Parking Lot, Meeting Room Scheduler, In-memory Cache, Elevator
Scoring	Deterministic coverage / graph / walkthrough / scope. Optional LLM for prose only
Auth	None

Header has Home, Problems, Attempts, and a day/night toggle.

Run locally

Node 20+.

```
npm install
npm test
npm run dev
```

<http://localhost:3000>

`OPENAI_API_KEY` is optional. Without it the checker still scores.

Pinned tests: Stall still covers assignment, god-class penalty, LLM throw → degraded success, evaluator crash → retry, revise does not mutate parent, follow-up seeds the prompt.

Architecture (short)

One Next.js process. Domain in `src/domain` does not import Next or `fs`. HTTP goes through `PracticeService` only.

```
src/  
  domain/      Attempt, Design, Problem, ports  
  catalog/     five briefs + examples  
  evaluation/  checker, optional LLM, hybrid  
  application/ PracticeService  
  infrastructure/ json store, clock, HTTP LLM  
  app/         UI + API
```

Ports: `ProblemRepository`, `AttemptRepository`, `Evaluator`, `LlmClient`.

Attempt: draft → submitted → evaluating → evaluated | evaluation_failed. Revise clones. Follow-up copies the design and writes the interviewer prompt into notes.

On Vercel the store is `/tmp/spoke`, so history can reset. Local `data/attempts.json` is the durable path.

AI usage

I used an assistant (chat + code help) while building this. Below are the design calls that actually changed the product, including the ones I said no to.

1. What the learner submits

Early suggestions looked like a code editor plus hidden tests.

I did not make that the main artifact. You submit a design someone can talk about: locked clarifying questions, types with responsibilities, relationships, a three-beat walkthrough, assumptions, one rejected alternative. Code is optional.

This brief is LLD, not “make Parking Lot compile”. Tests will pass a god class.

2. Scoring when two designs are both ok

Easy answers: a golden class list, or “ask an LLM for 1–10”.

I rejected both. Problems list capabilities. Some only turn on if you locked a clarification. A checker looks at coverage, graph smells, walkthrough, scope. The model may add comments. Coverage stays deterministic.

“You didn’t name it `ParkingSpot`” just trains people to copy blogs.

3. Model slow or missing

Suggestions were retries, circuit breakers, queues. Different assignment.

HTTP timeout around 10 seconds. Missing key / abort / junk JSON still completes with the checker (`degraded=true`). `evaluation_failed` only if `evaluate()` throws. Retry is one method on the same Attempt.

4. How many services

Splitting problem / attempt / evaluation services would be fake HLD for five problems. One app. The seams that matter are `Evaluator` and `AttemptRepository`.

5. Studio

First UI sketch was a dashboard: sidebar, purple, jump to class boxes.

I changed it to Clarify → Structure → Walkthrough → Defend. The graph is a view of the types, not a drawing tool.

What I still consider my calls

- Attempt is the aggregate. Evaluations don't get edited in place.
- Revise clones. History stays.
- Follow-up is a real next attempt.
- Signal matching is leaky. I wrote that down.
- No k8s, no contest clock, no fake login.

What the assistant touched

First drafts of the five briefs, a lot of the React studio, and the tests.

I rewrote the domain, `validateForSubmit`, `walkthrough/scope` checks, and `HybridEvaluator` when early versions were too close to a golden-name checker. Parking Lot example uses `Stall` on purpose.

If a design choice is in the design note, I can defend it without the chat log.

Other notes

Research: `submission/research-note.md` (also `RESEARCH.md` in the repo).

Design: `submission/design-note.md` (also `DESIGN.md`).